

Tarea 02 - Gestión de Eventos y Participantes con Spring Boot: Refactorización progresiva

José Manuel Sánchez Álvarez

Contents

0.1	Orden sugerido de refactorización	1
0.2	Resumen estructurado	3

0.1 Orden sugerido de refactorización

0.1.1 Fase 1 — Controlador “gordo” (punto de partida, realizado en tarea anterior)

El controlador hace todo:

- Accede al repositorio directamente.
- Contiene la lógica de negocio y validaciones básicas.
- Devuelve directamente `ResponseEntity` con entidades (`Evento`).

Objetivo: que funcione el CRUD completo antes de modularizar.

0.1.2 Fase 2 — Separación de responsabilidades

Mover lógica de negocio al servicio.

1. Crear la capa de servicio (`EventoService`):

- Define los métodos `listarEventos`, `obtenerEventoPorId`, `crearEvento`, `actualizarEvento`, `borrarEvento`.
- Pasa el repositorio como dependencia (inyección).

2. Controlador → Servicio:

- El controlador **solo delega** en el servicio.
- Retorna el resultado de cada método del servicio.
- Ya no usa el repositorio directamente.

3. Objetivo didáctico: comprender el principio “*Separation of Concerns*” y el papel del servicio como intermediario.

0.1.3 Fase 3 — Validaciones y excepciones

Refinar la capa de servicio.

1. Añadir validaciones en el servicio (por ejemplo, comprobar si el evento existe).
2. Crear una excepción personalizada (`EventoNotFoundException`).
3. Crear un manejador global de excepciones (`@ControllerAdvice`).

Así el controlador queda limpio y coherente con buenas prácticas REST.

0.1.4 Fase 4 — DTOs y conversión

Evitar exponer entidades directamente.

1. Crear `EventoDTO` o `EventoResponseDTO`.
2. Crear un *mapper* o conversión sencilla (`ModelMapper` o manual).
3. Actualizar el servicio/controlador para trabajar con DTOs.

Esto prepara el terreno para incluir filtrado, paginación y HATEOAS después.

0.1.5 Fase 5 — Paginación y ordenación

Integrar `Pageable`.

1. Cambiar el método de listar:

```
@GetMapping
public Page<EventoDTO> listarEventos(Pageable pageable)
```

2. En el servicio usar:

```
eventoRepository.findAll(pageable)
```

3. Probar URLs como:

```
/eventos?page=0&size=10&sort=fecha,asc
```

Aquí se introducen conceptos de eficiencia y escalabilidad.

0.1.6 Fase 6 — Filtrado avanzado y búsqueda

Añadir parámetros opcionales:

- `/eventos?nombre=Concierto`
- `/eventos?fechaInicio=2025-05-10`

Puedes hacerlo con `@RequestParam` o `Specification/Example`.

0.1.7 Fase 7 — HATEOAS y enlaces (no necesario, no penaliza)

Añadir hipermedios si estás mostrando cómo diseñar una API REST completa:

- Uso de `EntityModel`, `PagedModel`, `linkTo`, `methodOn`.
-

0.1.8 Fase 8 — Tests

Cuando ya está modularizado:

- Unit tests al servicio con `@MockBean` o `Mockito`.
 - Tests de integración al controlador (`@SpringBootTest`, `@AutoConfigureMockMvc`).
-

0.2 Resumen estructurado

Fase	Enfoque	Objetivo didáctico
1	Controlador completo	Tener base funcional
2	Crear capa de servicio	Separación de responsabilidades
3	Validaciones y excepciones	Manejo correcto de errores
4	DTOs	Buen diseño de API y encapsulación
5	Paginación / Ordenación	Escalabilidad
6	Filtrado	Consultas más útiles
7	HATEOAS	API REST enriquecida (no necesario)
8	Tests	Asegurar calidad y mantenimiento